Coding & Solution

Date	27 June 2026
Team ID	
Project Name	Credit Card Approval Prediction
Maximum Marks	5 Marks

Coding & Solution :

The **Credit Card Approval Prediction System** is developed using **Python, Flask, Scikit-learn, and XGBoost** to automate the credit card approval process. The application analyzes applicant details such as income, employment status, education level, credit history, and other financial information to predict whether a credit card application is likely to be **approved or rejected**.

The solution includes data preprocessing, feature engineering, machine learning model training, model evaluation, and deployment through a Flask web application. Multiple classification algorithms—**Logistic Regression, Decision Tree, Random Forest, and XGBoost**—are trained and compared to select the best-performing model. The trained model is saved using **Pickle** and integrated into the Flask application for real-time prediction.

The project follows a modular structure with separate components for data preprocessing, model training, prediction, and the web interface. Clean coding practices, meaningful variable names, proper comments, and error handling are implemented to improve readability, maintainability, and reusability. The complete source code, dataset, trained model, HTML templates, CSS files, and setup instructions are included in the project, and the code is maintained in a GitHub repository for version control and collaboration.

This solution provides a fast, accurate, and user-friendly platform for banks and financial institutions to make reliable credit card approval decisions while reducing manual effort and processing time.

Instructions:

1. Ensure the application is developed using **Python, Flask, Scikit-learn, and XGBoost**, following clean coding standards and proper project structure.
2. The solution should accurately predict whether a **credit card application will be approved or rejected** based on applicant details such as income, employment status, education level, credit history, and other financial information.
3. Include all required project files, including the **source code, trained machine learning model (model.pkl), dataset, HTML templates, CSS files, configuration files, and setup instructions** needed to run the application successfully.

4. Upload the complete project to a **GitHub repository** and provide the repository link in the Solution Summary section. Ensure the repository includes a README file with installation steps, required libraries, and execution instructions.

Solution Summary :

Field	Details
Repository Link / URL	https://github.com/makarajyothi-miriyala/AI-ML-and-GEN-AI-Track-Project-Template
Programming Language(s)	Python, HTML, CSS, JavaScript
Framework(s) Used	Flask, Scikit-learn, XGBoost
Key Features Implemented	• Data preprocessing and feature engineering • Machine learning model training using Logistic Regression, Decision Tree, Random Forest, and XGBoost • Best model selection and serialization using Pickle • Flask-based web application for real-time prediction • User-friendly interface for applicant data entry and prediction results • Model evaluation using Accuracy, Confusion Matrix, and Classification Report
Pending / Incomplete Features	IBM Watson cloud deployment (optional), prediction history dashboard, user authentication
Setup / Run Instructions	1. Install required libraries using pip install -r requirements.txt. 2. Run app.py using python app.py. 3. Open the provided localhost URL in a web browser. 4. Enter applicant details and click **Predict** to view the approval result.

Code Quality Checklist :

S.No	Criteria	Status (Yes / No)
1	Code is modular and organized into functions / classes	Yes
2	Meaningful variable and function names are used	Yes
3	Code includes comments / documentation where necessary	Yes
4	Error handling is implemented for critical operations	Yes
5	The application runs without critical errors	Yes
6	Code is committed to a version control repository	Yes

Additional Notes / Comments:

The project follows a modular architecture with separate components for data preprocessing, machine learning model training, and Flask web application development. Clean coding practices, reusable functions, and meaningful naming conventions have been used to improve readability, maintainability, and scalability. The solution successfully predicts whether a credit card application is likely to be **approved or rejected** based on applicant financial and demographic information.